

设计模式期末复习

软件模式

- 软件模式是将模式的一般概念应用于软件开发领域，即软件开发的总体指导思路或参照样板。软件模式并非仅限于设计模式，还包括架构模式、分析模式和过程模式等，实际上，在软件生存期的每一个阶段都存在着一些被认同的模式。
- 软件模式可以认为是对软件开发这一特定“问题”的“解法”的某种统一表示，软件模式等于一定条件下的出现的问题以及解法。软件模式的基础结构由 4 个部分构成：问题描述、前提条件（环境或约束条件）、解法和效果。

软件模式与具体的应用领域无关，在模式发现过程中需要遵循大三律 (Rule of Three)，即只有经过三个以上不同类型（或不同领域）的系统的校验，一个解决方案才能从候选模式升格为模式。

设计模式

设计模式 (Design Pattern) 是一套被反复使用、多数人知晓的、经过分类编目的、代码设计经验的总结，使用设计模式是为了可重用代码、让代码更容易被他人理解、保证代码可靠性。

设计模式一般有如下几个基本要素：模式名称、问题、目的、解决方案、效果、实例代码和相关设计模式，其中的关键元素包括以下四个方面：

- 模式名称
- 问题
- 解决方案
- 效果

根据其目的 (模式是用来做什么的) 可分为创建型 (Creational)、结构型 (Structural) 和行为型 (Behavioral) 三种：

- 创建型模式主要用于创建对象
- 结构型模式主要用于处理类或对象的组合
- 行为型模式主要用于描述对类或对象怎样交互和怎样分配职责

根据范围，即模式主要是用于处理类之间关系还是处理对象之间的关系，可分为类模式和对象模式两种：

- 类模式处理类和子类之间的关系，这些关系通过继承建立，在编译时刻就被确定下来，是属于静态的。
- 对象模式处理对象间的关系，这些关系在运行时刻变化，更具动态性。

设计原则

- 对于面向对象的软件系统设计来说，在支持可维护性的同时，需要提高系统的可复用性。
- 软件的复用可以提高软件的开发效率，提高软件质量，节约开发成本，恰当的复用还可以改善系统的可维护性。
- 单一职责原则要求在软件系统中，一个类只负责一个功能领域中的相应职责。
- 开闭原则要求一个软件实体应当对扩展开放，对修改关闭，即在不修改源代码的基础上扩展一个系统的行为。
- 里氏代换原则可以通俗表述为在软件中如果能够使用基类对象，那么一定能够使用其子类对象。
- 依赖倒转原则要求抽象不应该依赖于细节，细节应该依赖于抽象；要针对接口编程，不要针对实现编程。

- 接口隔离原则要求客户端不应该依赖那些它不需要的接口，即将一些大的接口细化成一些小的接口供客户端使用。
- 合成复用原则要求复用时尽量使用对象组合，而不使用继承。
- 迪米特法则要求一个软件实体应当尽可能少的与其他实体发生相互作用。

表驱动法

表驱动法 (Table-Driven Approach) 是一种软件设计方法，旨在通过使用预定义的表格或映射来处理问题。它的目标是提高代码的可读性、可维护性和可扩展性。

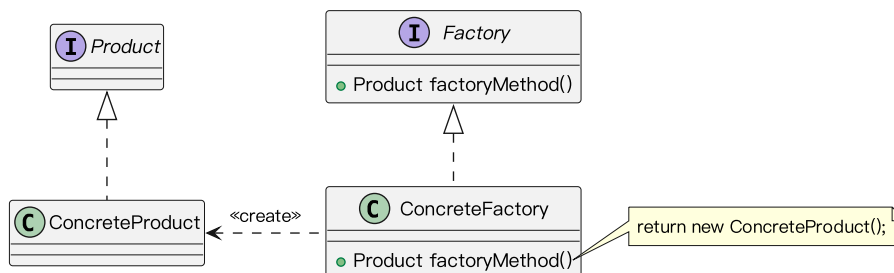
表驱动法的原理是将问题的逻辑和决策转化为数据结构，通过查表的方式来获取相应的结果或执行相应的操作。它基于数据驱动的思想，将复杂的条件判断和逻辑分支转化为简单的数据查找，从而减少代码的复杂性和冗余性。

如何快速从表中查询条目

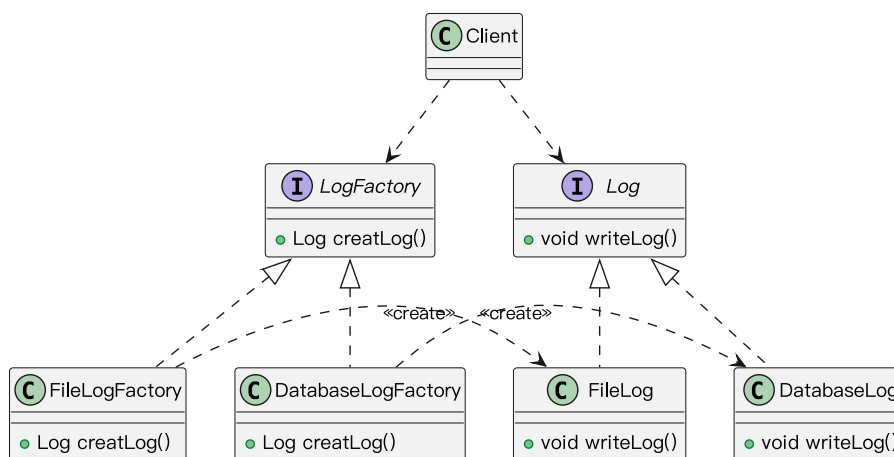
- 直接访问
- 索引访问
- 阶梯访问

工厂方法模式

工厂方法模式 (Factory Method Pattern) 又称为工厂模式，也叫虚拟构造器 (Virtual Constructor) 模式或者多态工厂 (Polymorphic Factory) 模式，它属于类创建型模式。在工厂方法模式中，工厂父类负责定义创建产品对象的公共接口，而工厂子类则负责生成具体的产品对象，这样做的目的是将产品类的实例化操作延迟到工厂子类中完成，即通过工厂子类来确定究竟应该实例化哪一个具体产品类。



日志记录器：某系统日志记录器要求支持多种日志记录方式，如文件记录、数据库记录等，且用户可以根据要求动态选择日志记录方式，现使用工厂方法模式设计该系统。



工厂方法模式的优点

- 在工厂方法模式中，工厂方法用来创建客户所需要的产品，同时还向客户隐藏了哪种具体产品类将被实例化这一细节，用户只需要关心所需产品对应的工厂，无须关心创建细节，甚至无须知道具体产品类的类名。
- 基于工厂角色和产品角色的多态性设计是工厂方法模式的关键。它能够使工厂可以自主确定创建何种产品对象，而如何创建这个对象的细节则完全封装在具体工厂内部。工厂方法模式之所以又被称为多态工厂模式，是因为所有的具体工厂类都具有同一抽象父类。
- 使用工厂方法模式的另一个优点是在系统中加入新产品时，无须修改抽象工厂和抽象产品提供的接口，无须修改客户端，也无须修改其他的具体工厂和具体产品，而只要添加一个具体工厂和具体产品就可以了。这样，系统的可扩展性也就变得非常好，完全符合“开闭原则”。

工厂方法模式的缺点

- 在添加新产品时，需要编写新的具体产品类，而且还要提供与之对应的具体工厂类，系统中类的个数将成对增加，在一定程度上增加了系统的复杂度，有更多的类需要编译和运行，会给系统带来一些额外的开销。

- 由于考虑到系统的可扩展性，需要引入抽象层，在客户端代码中均使用抽象层进行定义，增加了系统的抽象性和理解难度，且在实现时可能需要用到 DOM、反射等技术，增加了系统的实现难度。

在以下情况下可以使用工厂方法模式：

- 一个类不知道它所需要的对象的类：在工厂方法模式中，客户端不需要知道具体产品类的类名，只需要知道所对应的工厂即可，具体的产品对象由具体工厂类创建；客户端需要知道创建具体产品的工厂类。
- 一个类通过其子类来指定创建哪个对象：在工厂方法模式中，对于抽象工厂类只需要提供一个创建产品的接口，而由其子类来确定具体要创建的对象，利用面向对象的多态性和里氏代换原则，在程序运行时，子类对象将覆盖父类对象，从而使系统更容易扩展。
- 将创建对象的任务委托给多个工厂子类中的某一个，客户端在使用时可以无须关心是哪一个工厂子类创建产品子类，需要时再动态指定，可将具体工厂类的类名存储在配置文件或数据库中。

与设计原则间的关系：

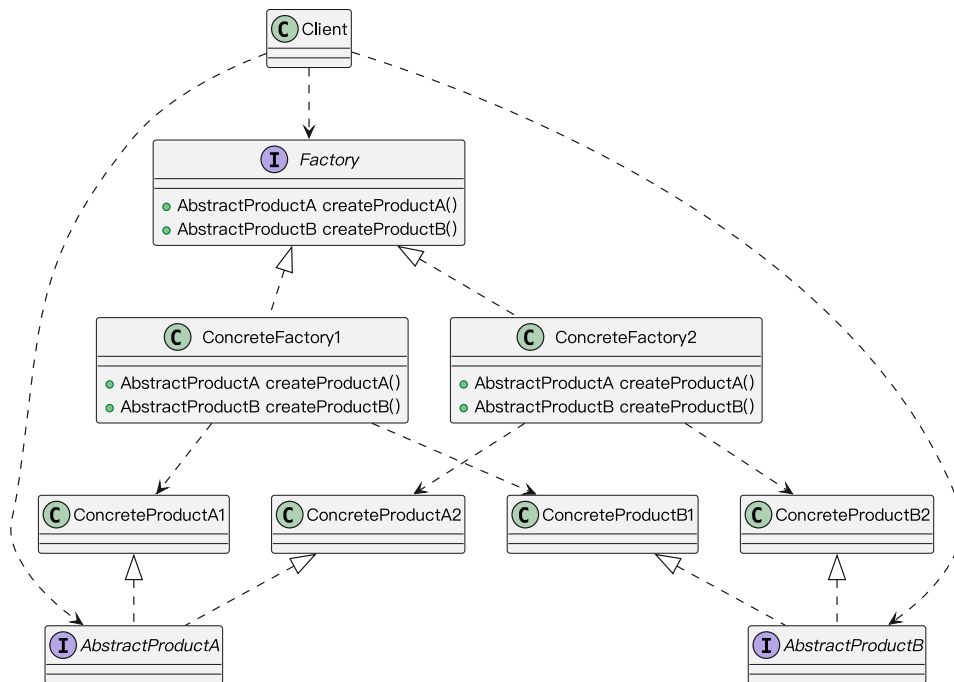
- 开闭原则（Open-Closed Principle）：工厂方法模式遵循开闭原则，通过将对象的创建委托给子类来实现对象的创建，而不是直接在客户端代码中实例化具体的对象。这样，当需要添加新的产品时，只需创建一个新的具体子类即可，而不需要修改现有的客户端代码。
- 单一职责原则（Single Responsibility Principle）：工厂方法模式将对象的创建职责封装在一个工厂类中，使得每个类都只负责单一的职责。工厂类负责创建对象，而具体的产品类负责实现产品的功能。
- 依赖倒置原则（Dependency Inversion Principle）：工厂方法模式通过依赖抽象，客户端代码只依赖于抽象的产品接口，而不依赖于具体的产品类。这样可以降低客户端与具体产品之间的耦合度，提高代码的灵活性和可维护性。

抽象工厂模式

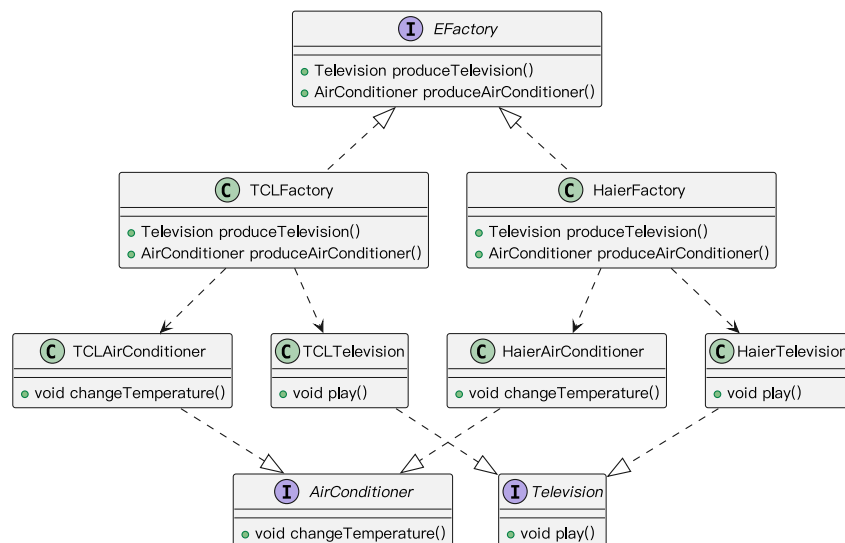
抽象工厂模式 (Abstract Factory Pattern): 提供一个创建一系列相关或相互依赖对象的接口, 而无须指定它们具体的类。抽象工厂模式又称为 Kit 模式, 属于对象创建型模式。

抽象工厂模式包含如下角色:

- AbstractFactory: 抽象工厂
- AbstractProduct: 抽象产品
- ConcreteFactory: 具体工厂
- Product: 具体产品



电器工厂: 一个电器工厂可以产生多种类型的电器, 如海尔工厂可以生产海尔电视机、海尔空调等, TCL 工厂可以生产 TCL 电视机、TCL 空调等, 相同品牌的电器构成一个产品族, 而相同类型的电器构成了一个产品等级结构, 现使用抽象工厂模式模拟该场景。



抽象工厂模式的优点:

- 抽象工厂模式隔离了具体类的生成，使得客户并不需要知道什么被创建。由于这种隔离，更换一个具体工厂就变得相对容易。所有的具体工厂都实现了抽象工厂中定义的那些公共接口，因此只需改变具体工厂的实例，就可以在某种程度上改变整个软件系统的行为。另外，应用抽象工厂模式可以实现高内聚低耦合的设计目的，因此抽象工厂模式得到了广泛的应用。
- 当一个产品族中的多个对象被设计成一起工作时，它能够保证客户端始终只使用同一个产品族中的对象。这对一些需要根据当前环境来决定其行为的软件系统来说，是一种非常实用的设计模式。
- 增加新的具体工厂和产品族很方便，无须修改已有系统，符合“开闭原则”。

抽象工厂模式的缺点：

- 在添加新的产品对象时，难以扩展抽象工厂来生产新种类的产品，这是因为在抽象工厂角色中规定了所有可能被创建的产品集合，要支持新种类的产品就意味着要对该接口进行扩展，而这将涉及到对抽象工厂角色及其所有子类的修改，显然会带来较大的不便。
- 上述缺点可概括为：增加新的工厂和产品族容易，增加新的产品等级结构麻烦（开闭原则的倾斜性）。

在以下情况下可以使用抽象工厂模式：

- 一个系统不应当依赖于产品类实例如何被创建、组合和表达的细节，这对于所有类型的工厂模式都是重要的。
- 系统中有多于一个的产品族，而每次只使用其中某一产品族。
- 属于同一个产品族的产品将在一起使用，这一约束必须在系统的设计中体现出来。
- 系统提供一个产品类的库，所有的产品以同样的接口出现，从而使客户端不依赖于具体实现。

与设计原则的关系：

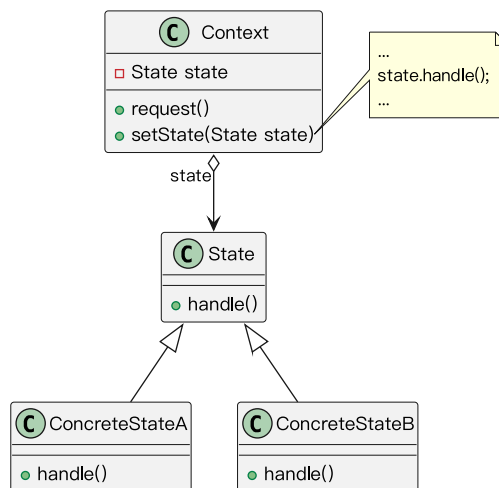
- 开闭原则（Open-Closed Principle）：抽象工厂模式遵循开闭原则，通过将对象的创建委托给具体的工厂类来实现对象的创建，而不是直接在客户端代码中实例化具体的对象。这样，在需要添加新的产品族时，只需创建新的具体工厂类，而不需要修改现有的客户端代码。
- 单一职责原则（Single Responsibility Principle）：抽象工厂模式将对象的创建职责封装在一个工厂接口中，每个具体工厂类负责创建一系列相关的产品。这样，每个具体工厂类都只负责单一的职责。
- 依赖倒置原则（Dependency Inversion Principle）：抽象工厂模式通过依赖抽象，客户端代码只依赖于抽象的工厂接口和产品接口，而不依赖于具体的工厂类和产品类。这样可以降低客户端与具体实现之间的耦合度，提高代码的灵活性和可维护性。
- 接口隔离原则（Interface Segregation Principle）：抽象工厂模式定义了工厂接口和产品接口，将产品的创建和使用解耦。每个产品接口只包含与该产品相关的方法，避免了客户端依赖于不需要的接口。

状态模式

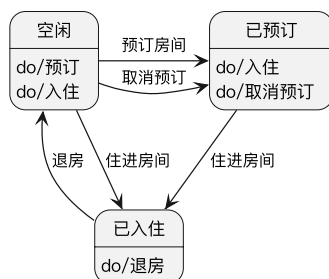
状态模式 (State Pattern): 允许一个对象在其内部状态改变时改变它的行为, 对象看起来似乎修改了它的类。其别名为状态对象 (Objects for States), 状态模式是一种对象行为型模式。

状态模式包含如下角色:

- Context: 环境类
- State: 抽象状态类
- ConcreteState: 具体状态类



房间预订状态转换图

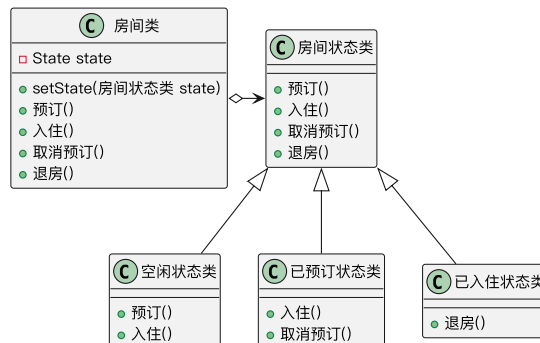


“空闲状态类”示例代码

```

...
if (预订房间) {
    预订操作;
    context.setState(new 已预订状态类());
} else if (住进房间) {
    入住操作;
    context.setState(new 已入住状态类());
}
...
  
```

使用状态模式构造的类图



状态模式的优点:

- 封装了转换规则。
- 枚举可能的状态, 在枚举状态之前需要确定状态种类。
- 将所有与某个状态有关的行为放到一个类中, 并且可以方便地增加新的状态, 只需要改变对象状态即可改变对象的行为。
- 允许状态转换逻辑与状态对象合成一体, 而不是某一个巨大的条件语句块。
- 可以让多个环境对象共享一个状态对象, 从而减少系统中对象的个数。

状态模式的缺点:

- 状态模式的使用必然会增加系统类和对象的个数。
- 状态模式的结构与实现都较为复杂，如果使用不当将导致程序结构和代码的混乱。
- 状态模式对“开闭原则”的支持并不太好，对于可以切换状态的状态模式，增加新的状态类需要修改那些负责状态转换的源代码，否则无法切换到新增状态；而且修改某个状态类的行为也需修改对应类的源代码。

在以下情况下可以使用状态模式：

- 对象的行为依赖于它的状态（属性）并且可以根据它的状态改变而改变它的相关行为。
- 代码中包含大量与对象状态有关的条件语句，这些条件语句的出现，会导致代码的可维护性和灵活性变差，不能方便地增加和删除状态，使客户类与类库之间的耦合增强。在这些条件语句中包含了对象的行为，而且这些条件对应于对象的各种状态。

与设计原则的关系：

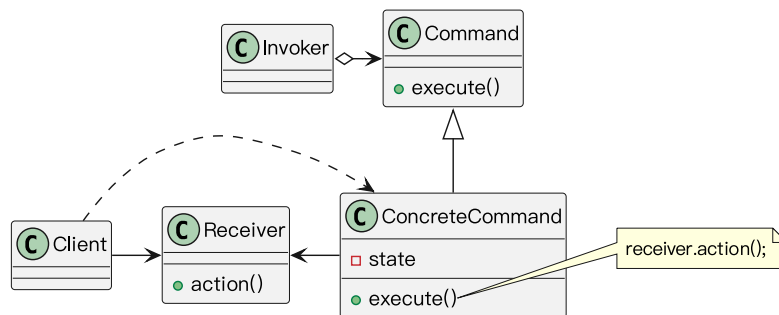
- 开闭原则（Open-Closed Principle）：状态模式遵循开闭原则，通过将对象的不同状态封装成独立的类，使得可以在不修改原有代码的情况下引入新的状态。这样可以方便地扩展和添加新的状态，而不会对原有代码产生影响。
- 单一职责原则（Single Responsibility Principle）：状态模式将对象的每个状态都封装成一个独立的类，使每个类都具有单一的职责，即表示特定的状态和定义相关的行为。这样可以使代码更加清晰、可读性更高，并且每个类可以专注于自己的职责。
- 依赖倒置原则（Dependency Inversion Principle）：状态模式通过定义抽象的状态类和上下文类之间的关系，使得上下文类依赖于抽象的状态类，而不是具体的状态类。这样可以降低上下文类与具体状态类之间的耦合度，使系统更加灵活和可扩展。
- 迪米特法则（Law of Demeter）：状态模式通过封装不同的状态和行为，使得上下文类不需要知道具体的状态类的细节。上下文类只需要和抽象的状态类进行交互，而不需要与所有的具体状态类直接交互。这符合迪米特法则的思想，减少了对象之间的耦合，提高了代码的可维护性和可扩展性。

命令模式

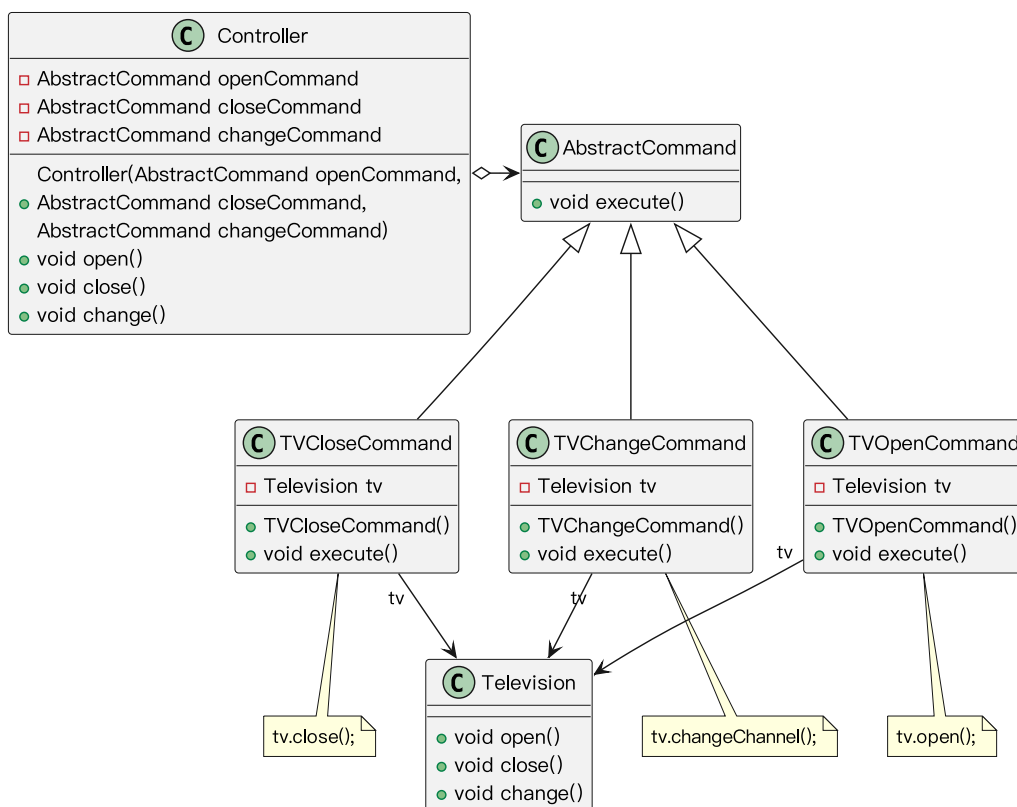
命令模式 (Command Pattern): 将一个请求封装为一个对象, 从而使我们可用不同的请求对客户进行参数化; 对请求排队或者记录请求日志, 以及支持可撤销的操作。命令模式是一种对象行为型模式, 其别名为动作 (Action) 模式或事务 (Transaction) 模式。

命令模式包含如下角色:

- Command: 抽象命令类
- ConcreteCommand: 具体命令类
- Invoker: 调用者
- Receiver: 接收者
- Client: 客户类



电视机遥控器: 电视机是请求的接收者, 遥控器是请求的发送者, 遥控器上有一些按钮, 不同的按钮对应电视机的不同操作。抽象命令角色由一个命令接口来扮演, 有三个具体的命令类实现了抽象命令接口, 这三个具体命令类分别代表三种操作: 打开电视机、关闭电视机和切换频道。显然, 电视机遥控器就是一个典型的命令模式应用实例。



命令模式的优点:

- 降低系统的耦合度。
- 新的命令可以很容易地加入到系统中。
- 可以比较容易地设计一个命令队列和宏命令（组合命令）。
- 可以方便地实现对请求的 Undo 和 Redo。

命令模式的缺点：

- 使用命令模式可能会导致某些系统有过多的具体命令类。因为针对每一个命令都需要设计一个具体命令类，因此某些系统可能需要大量具体命令类，这将影响命令模式的使用。

在以下情况下可以使用命令模式：

- 系统需要将请求调用者和请求接收者解耦，使得调用者和接收者不直接交互。
- 系统需要在不同的时间指定请求、将请求排队和执行请求。
- 系统需要支持命令的撤销 (Undo) 操作和恢复 (Redo) 操作。
- 系统需要将一组操作组合在一起，即支持宏命令。

与设计原则的关系：

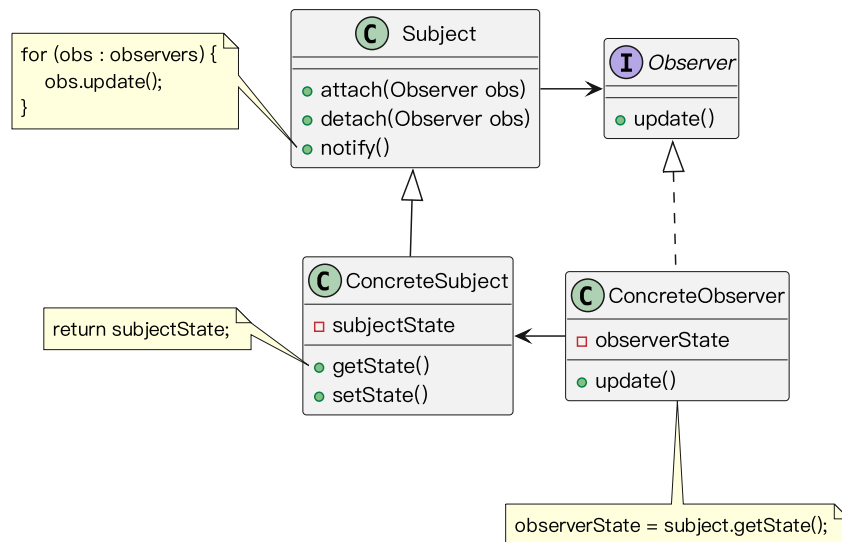
- 开闭原则（Open-Closed Principle）：命令模式遵循开闭原则，通过将请求封装成一个具体命令对象，可以在不修改现有代码的情况下新增不同的命令。这样可以方便地扩展和添加新的命令，而不会对原有代码产生影响。
- 单一职责原则（Single Responsibility Principle）：命令模式将每个请求封装成一个独立的命令对象，使得每个命令对象都具有单一的职责，即负责执行一个具体的请求操作。这样可以使代码更加清晰、可读性更高，并且每个命令对象可以专注于自己的职责。
- 依赖倒置原则（Dependency Inversion Principle）：命令模式通过将请求发送者与接收者解耦，使得请求发送者只依赖于抽象的命令接口，而不依赖于具体的命令对象和接收者对象。这样可以降低发送者和接收者之间的耦合度，提高代码的灵活性和可维护性。
- 迪米特法则（Law of Demeter）：命令模式通过将请求封装成一个具体的命令对象，使得发送者只与命令对象进行交互，而不需要直接与接收者进行交互。这符合迪米特法则的思想，减少了对象之间的耦合，使系统更加灵活和可扩展。

观察者模式

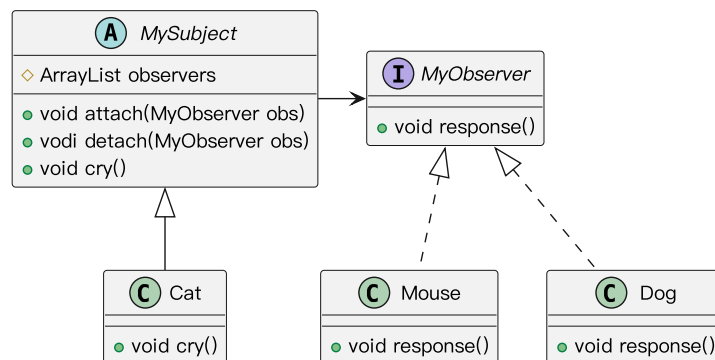
观察者模式 (Observer Pattern): 定义对象间的一种一对多依赖关系, 使得每当一个对象状态发生改变时, 其相关依赖对象皆得到通知并被自动更新。观察者模式又叫做发布-订阅 (Publish/Subscribe) 模式、模型-视图 (Model/View) 模式、源-监听器 (Source/Listener) 模式或从属者 (Dependents) 模式。观察者模式是一种对象行为型模式。

观察者模式包含如下角色:

- Subject: 目标
- ConcreteSubject: 具体目标
- Observer: 观察者
- ConcreteObserver: 具体观察者



猫、狗与老鼠: 假设猫是老鼠和狗的观察目标, 老鼠和狗是观察者, 猫叫老鼠跑, 狗也跟着叫, 使用观察者模式描述该过程。



观察者模式的优点:

- 观察者模式可以实现表示层和数据逻辑层的分离, 并定义了稳定的消息更新传递机制, 抽象了更新接口, 使得可以有各种各样不同的表示层作为具体观察者角色。
- 观察者模式在观察目标和观察者之间建立一个抽象的耦合。
- 观察者模式支持广播通信。
- 观察者模式符合开闭原则的要求。

观察者模式的缺点：

- 如果一个观察目标对象有很多直接和间接的观察者的话，**将所有的观察者都通知到会花费很多时间**。
- 如果在观察者和观察目标之间有**循环依赖**的话，观察目标会触发它们之间进行循环调用，**可能导致系统崩溃**。
- 观察者模式**没有相应的机制让观察者知道所观察的目标对象是怎么发生变化的**，而仅仅只是知道观察目标发生了变化。

在以下情况下可以使用观察者模式：

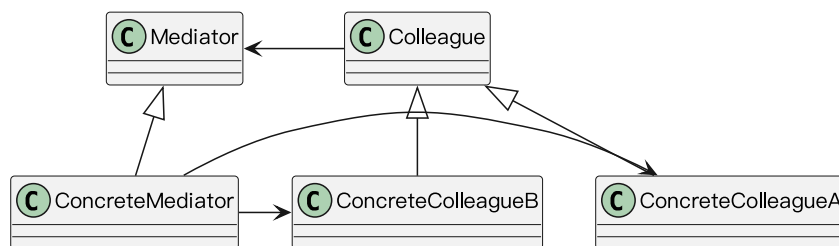
- 一个抽象模型有两个方面，其中一个方面依赖于另一个方面。将这些方面封装在独立的对象中使它们可以各自独立地改变和复用。
- 一个对象的改变将导致其他一个或多个对象也发生改变，而不知道具体有多少对象将发生改变，可以降低对象之间的耦合度。
- 一个对象必须通知其他对象，而并不知道这些对象是谁。需要在系统中创建一个触发链，A 对象的行为将影响 B 对象，B 对象的行为将影响 C 对象……，可以使用观察者模式创建一种链式触发机制。

中介者模式

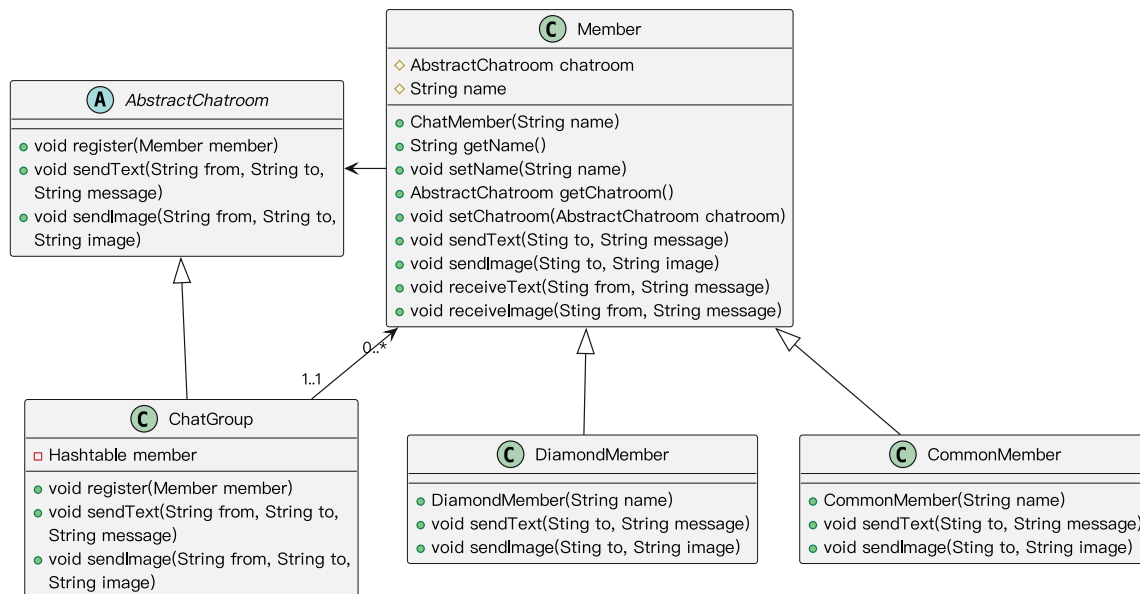
中介者模式 (Mediator Pattern) 定义：用一个中介对象来封装一系列的对象交互，中介者使各对象不需要显式地相互引用，从而使其耦合松散，而且可以独立地改变它们之间的交互。中介者模式又称为调停者模式，它是一种对象行为型模式。

中介者模式包含如下角色：

- Mediator：抽象中介者
- ConcreteMediator：具体中介者
- Colleague：抽象同事类
- ConcreteColleague：具体同事类



虚拟聊天室：某论坛系统欲增加一个虚拟聊天室，允许论坛会员通过该聊天室进行信息交流，普通会员 (CommonMember) 可以给其他会员发送文本信息，钻石会员 (DiamondMember) 既可以给其他会员发送文本信息，还可以发送图片信息。该聊天室可以对不雅字符进行过滤，如“日”等字符；还可以对发送的图片大小进行控制。用中介者模式设计该虚拟聊天室。



中介者模式的优点：

- 简化了对象之间的交互。
- 减少子类生成。
- 将各同事解耦。
- 可以简化各同事类的设计和实现。

中介者模式的缺点：

- 在具体中介者类中包含了同事之间的交互细节，可能会导致具体中介者类非常复杂，使得系统难以维护。

在以下情况下可以使用中介者模式：

- 系统中对象之间存在复杂的引用关系，产生的相互依赖关系结构混乱且难以理解。
- 一个对象由于引用了其他很多对象并且直接和这些对象通信，导致难以复用该对象。
- 想通过一个中间类来封装多个类中的行为，而又不想生成太多的子类。可以通过引入中介者类来实现，在中介者中定义对象交互的公共行为，如果需要改变行为则可以增加新的中介者类。

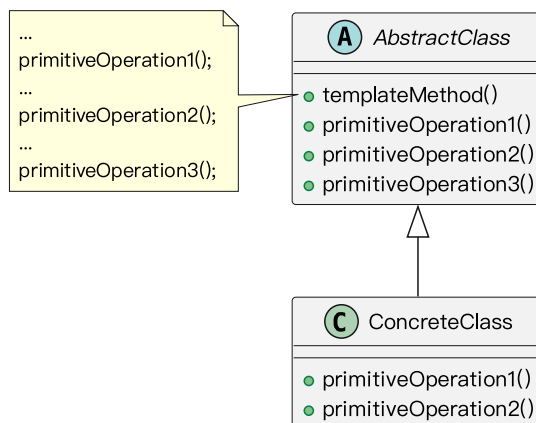
模板方法模式

模板方法模式 (Template Method Pattern): 定义一个操作中算法的骨架, 而将一些步骤延迟到子类中, 模板方法使得子类可以不改变一个算法的结构即可重定义该算法的某些特定步骤。模板方法是一种类行为型模式。

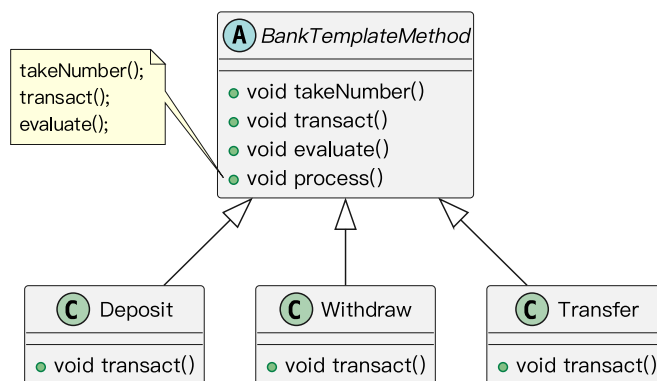
模板方法模式包含如下角色:

- AbstractClass: 抽象类

- ConcreteClass: 具体子类



银行业务办理流程: 在银行办理业务时, 一般都包含几个基本步骤, 首先需要取号排队, 然后办理具体业务, 最后需要对银行工作人员进行评分。无论具体业务是取款、存款还是转账, 其基本流程都一样。现使用模板方法模式模拟银行业务办理流程。



模板方法模式的优点:

- 模板方法模式在一个类中抽象地定义算法, 而由它的子类实现细节的处理。
- 模板方法模式是一种代码复用的基本技术。
- 模板方法模式导致一种反向的控制结构, 通过一个父类调用其子类的操作, 通过对子类的扩展增加新的行为, 符合“开闭原则”。

模板方法模式的缺点:

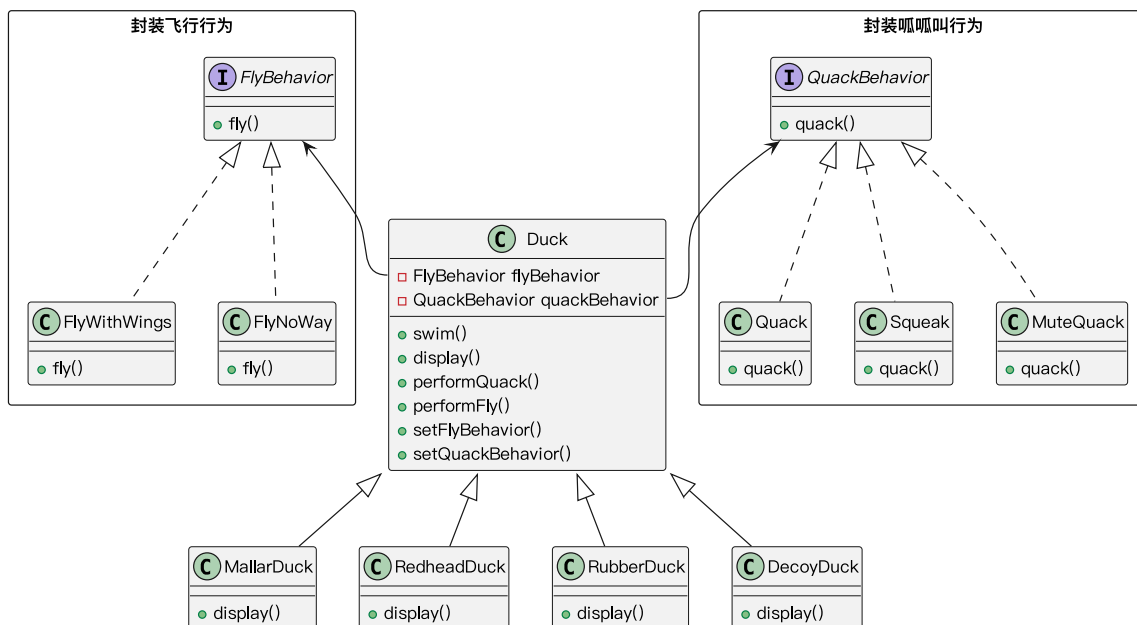
- 每个不同的实现都需要定义一个子类, 这会导致类的个数增加, 系统更加庞大, 设计也更加抽象, 但是更加符合“单一职责原则”, 使得类的内聚性得以提高。

在以下情况下可以使用模板方法模式:

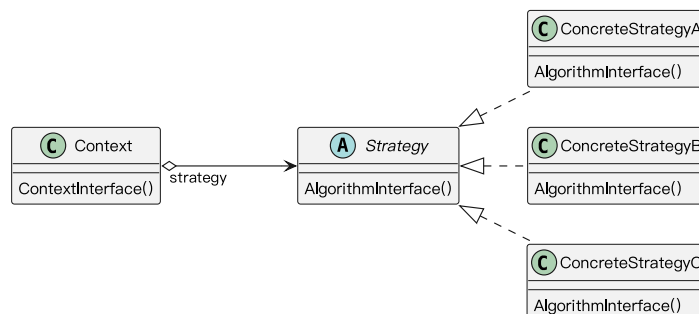
- 一次性实现一个算法的不变的部分，并将可变的行为留给子类来实现。
- 各子类中公共的行为应被提取出来并集中到一个公共父类中以避免代码重复。
- 对一些复杂的算法进行分割，将其算法中固定不变的部分设计为模板方法和父类具体方法，而一些可以改变的细节由其子类来实现。
- 控制子类的扩展。

策略模式

策略模式 (Strategy Pattern) 定义了一系列算法，分别封装起来，并使它们可替换。此模式让算法的变化独立于使用算法的客户。



将文本流分成几行。由于存在许多用于将文本流分成行的算法，将所有这样的算法硬连接到类中是不可取的。



策略模式的使用场景

- 许多相关的类仅在**行为**上有所不同，策略提供了一种使用多种行为之一配置类的方法
- 需要**算法的不同变体**。例如，您定义了反映不同空间/时间权衡的算法，将这些变体实现为算法的类层次结构时，可以使用策略。
- 一种算法使用客户端不应该知道的数据。使用策略模式可**避免暴露复杂的、特定于算法的数据结构**。
- 一个类定义了许多行为，这些行为在其操作中显示为多个条件语句。此时可以代替这些条件，将相关的条件分支移到他们自己的**策略类**中。

策略模式使用结果：

- 定义了相关算法家族。策略类的层次结构定义了一系列算法或行为，以供上下文重用。继承可以帮助排除算法的通用功能。
- 子类化的替代方法。

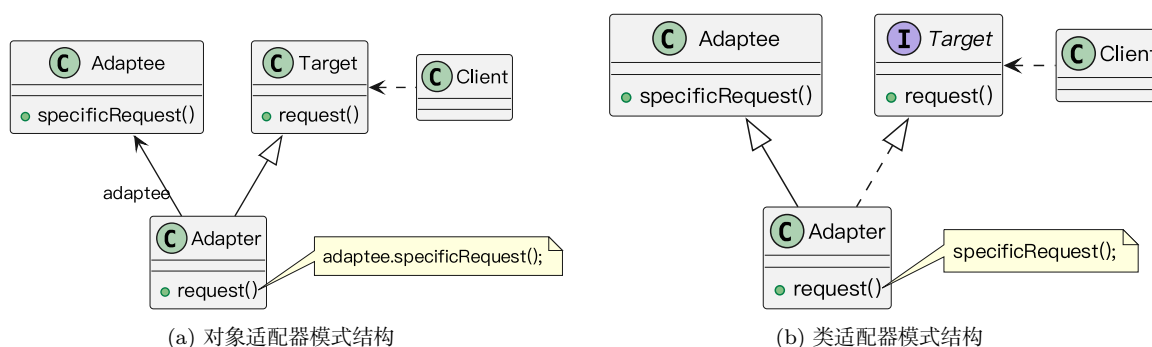
- 策略消除条件语句。
- 多种实现方式。策略可以提供相同行为的不同实现。客户可以选择具有不同时间和空间权衡的策略。
- 客户必须意识到不同的策略。这种模式有一个潜在的缺点，即**客户在选择合适的策略之前必须先了解策略的不同**，不然客户可能会遇到实现问题。
- 策略和上下文之间的通信开销。
- 对象数量增加。
- 模式一般都会有的缺点：
 - 增加设计的复杂度和增加类的个数（增加辅助类）
 - 增加隔阂和方法调用，降低软件运行的效率，但是已经不是目前主要的问题
- Java 的加密方法、时间显示算法等都是通过策略模式实现的。

适配器模式

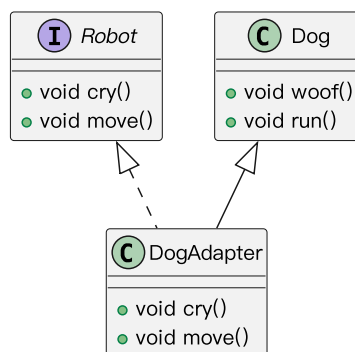
适配器模式 (Adapter Pattern): 将一个接口转换成客户希望的另一个接口, 适配器模式使接口不兼容的那些类可以一起工作, 其别名为包装器 (Wrapper)。适配器模式既可以作为类结构型模式, 也可以作为对象结构型模式。

适配器模式包含如下角色:

- Target: 目标抽象类
- Adapter: 适配器类
- Adaptee: 适配者类
- Client: 客户类



仿生机器人: 现需要设计一个可以模拟各种动物行为的机器人, 在机器人中定义了一系列方法, 如机器人叫喊方法 `cry()`、机器人移动方法 `move()` 等。如果希望在不修改已有代码的基础上使得机器人能够像狗一样叫, 像狗一样跑, 使用适配器模式进行系统设计。



适配器模式的优点:

- 将目标类和适配者类解耦, 通过引入一个适配器类来重用现有的适配者类, 而无须修改原有代码。
- 增加了类的透明性和复用性, 将具体的实现封装在适配器类中, 对于客户端类来说是透明的, 而且提高了适配者的复用性。
- 灵活性和扩展性都非常好, 通过使用配置文件, 可以很方便地更换适配器, 也可以在不修改原有代码的基础上增加新的适配器类, 完全符合“开闭原则”。

类适配器模式还具有如下优点:

- 由于适配器类是适配者类的子类, 因此可以在适配器类中置换一些适配者的方法, 使得适配器的灵活性更强。

对象适配器模式还具有如下优点:

- 一个对象适配器可以把多个不同的适配者适配到同一个目标，也就是说，**同一个适配器可以把适配者类和它的子类都适配到目标接口。**

类适配器模式的缺点如下：

- 对于 Java、C# 等不支持多重继承的语言，一次最多只能适配一个适配者类，而且目标抽象类只能为抽象类，不能为具体类，**其使用有一定的局限性**，不能将一个适配者类和它的子类都适配到目标接口。

在以下情况下可以使用适配器模式：

- 系统需要使用现有的类，而这些类的接口不符合系统的需要。
- 想要建立一个可以重复使用的类，用于与一些彼此之间没有太大关联的一些类，包括一些可能在将来引进的类一起工作。

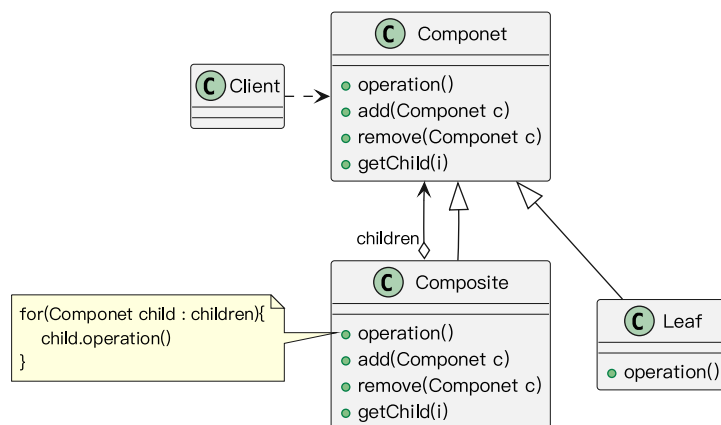
组合模式

组合模式 (Composite Pattern): 组合多个对象形成树形结构以表示“整体-部分”的结构层次。组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性。

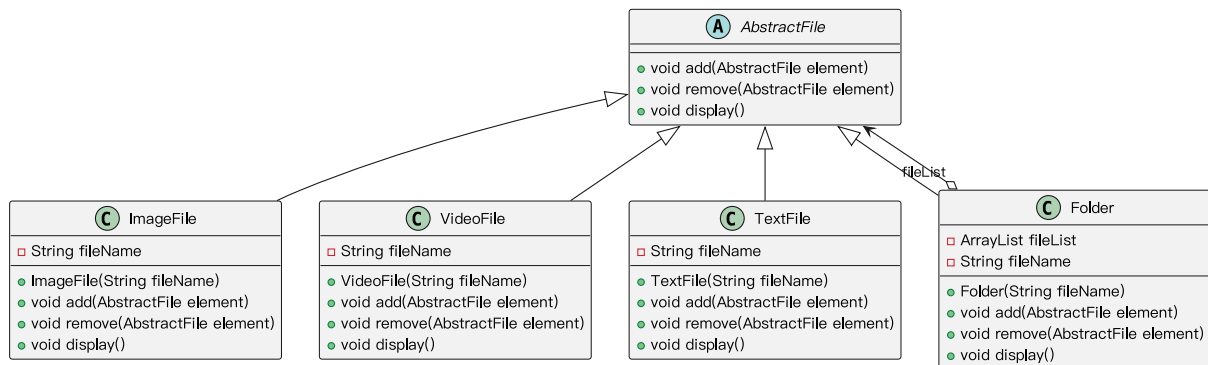
组合模式又可以称为整体-部分 (Part-Whole) 模式，属于对象的结构模式，它将对象组织到树结构中，可以用来描述整体与部分的关系。

组合模式包含如下角色：

- Component：抽象构件
- Composite：容器构件
- Leaf：叶子构件
- Client：客户类



文件浏览：文件有不同类型，不同类型的文件其浏览方式有所区别，如文本文件和图片文件的浏览方式就不相同。对文件夹的浏览实际上就是对其中所包含文件的浏览，而客户端可以一致地对文件和文件夹进行操作，无须关心它们的区别。使用组合模式来模拟文件的浏览操作。



组合模式的优点：

- 可以清楚地定义分层次的复杂对象，表示对象的全部或部分层次，使得增加新构件也更容易。
- 客户端调用简单，客户端可以一致的使用组合结构或其中单个对象。
- 定义了包含叶子对象和容器对象的类层次结构，叶子对象可以被组合成更复杂的容器对象，而这个容器对象又可以被组合，这样不断递归下去，可以形成复杂的树形结构。
- 更容易在组合体内加入对象构件，客户端不必因为加入了新的对象构件而更改原有代码。组合模式

组合模式的缺点：

- 使设计变得更加抽象，对象的业务规则如果很复杂，则实现组合模式具有很大挑战性，而且不是所有的方法都与叶子对象子类都有关联。
- 增加新构件时可能会产生一些问题，很难对容器中的构件类型进行限制。

在以下情况下可以使用组合模式：

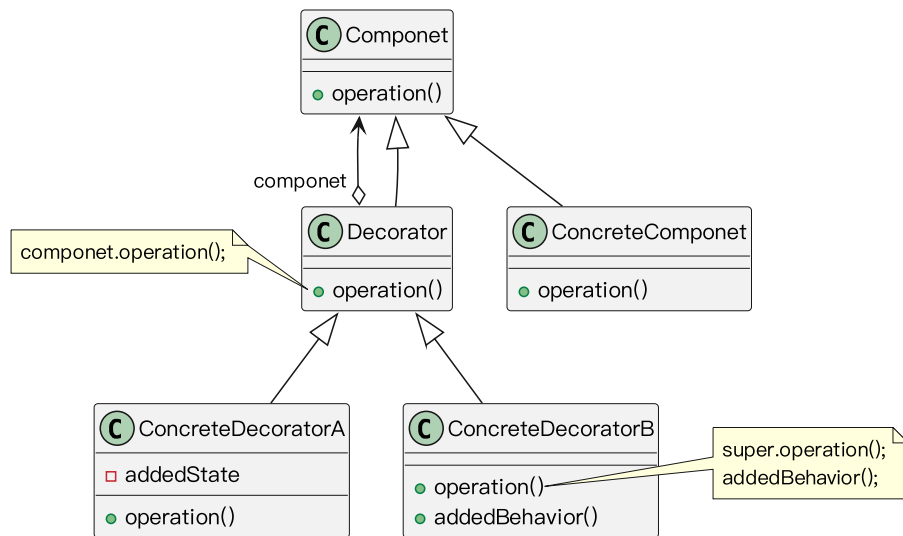
- 需要表示一个**对象整体或部分层次**，在具有整体和部分的层次结构中，希望通过一种方式忽略整体与部分的差异，可以一致地对待它们。
- 让客户能够忽略不同对象层次的变化，客户端可以**针对抽象构件编程**，无须关心对象层次结构的细节。
- 对象的结构是动态的并且复杂程度不一样，但客户需要**一致地处理它们**。

装饰模式

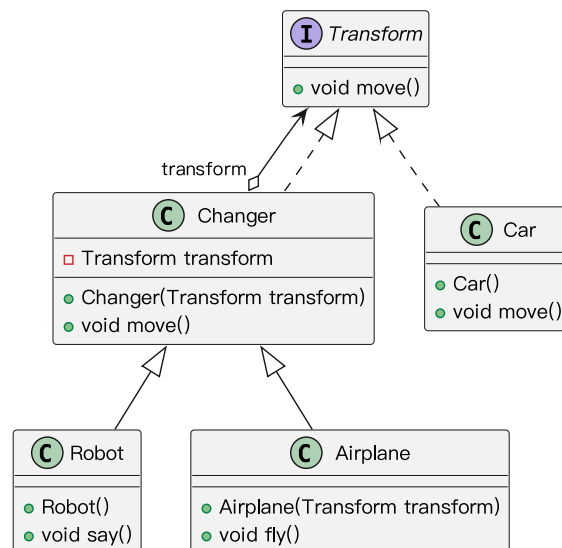
装饰模式 (Decorator Pattern)：动态地给一个对象增加一些额外的职责 (Responsibility)，就增加对象功能来说，装饰模式比生成子类实现更为灵活。其别名也可以称为包装器 (Wrapper)，与适配器模式的别名相同，但它们适用于不同的场合。根据翻译的不同，装饰模式也有人称之为“油漆工模式”，它是一种对象结构型模式。

装饰模式包含如下角色：

- Component：抽象构件
- ConcreteComponent：具体构件
- Decorator：抽象装饰类
- ConcreteDecorator：具体装饰类



变形金刚：变形金刚在变形之前是一辆汽车，它可以在陆地上移动。当它变成机器人之后除了能够在陆地上移动之外，还可以说话；如果需要，它还可以变成飞机，除了在陆地上移动还可以在天空中飞翔。



装饰模式的优点：

- 装饰模式与继承关系的目的都是要扩展对象的功能，但是装饰模式可以提供比继承更多的灵活性。

- 可以通过一种动态的方式来扩展一个对象的功能，通过配置文件可以在运行时选择不同的装饰器，从而实现不同的行为。
- 通过使用不同的具体装饰类以及这些装饰类的排列组合，可以创造出很多不同行为的组合。可以使用多个具体装饰类来装饰同一对象，得到功能更为强大的对象。
- 具体构件类与具体装饰类可以独立变化，用户可以根据需要增加新的具体构件类和具体装饰类，在使用时再对其进行组合，原有代码无须改变，符合“开闭原则”。

装饰模式的缺点：

- 使用装饰模式进行系统设计时将产生很多小对象，这些对象的区别在于它们之间相互连接的方式有所不同，而不是它们的类或者属性值有所不同，同时还将产生很多具体装饰类。这些装饰类和小对象的产生将增加系统的复杂度，加大学习与理解的难度。
- 这种比继承更加灵活机动的特性，也同时意味着装饰模式比继承更加易于出错，排错也很困难，对于多次装饰的对象，调试时寻找错误可能需要逐级排查，较为烦琐。

在以下情况下可以使用装饰模式：

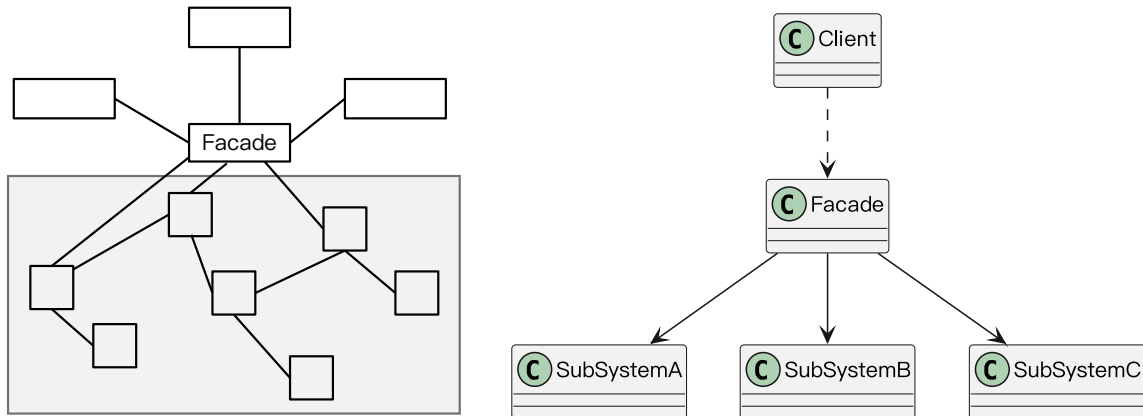
- 在不影响其他对象的情况下，以动态、透明的方式给单个对象添加职责。
- 需要动态地给一个对象增加功能，这些功能也可以动态地被撤销。
- 当不能采用继承的方式对系统进行扩充或者采用继承不利于系统扩展和维护时。不能采用继承的情况主要有两类：第一类是系统中存在大量独立的扩展，为支持每一种组合将产生大量的子类，使得子类数目呈爆炸性增长；第二类是因为类定义不能继承（如 final 类）。

外观模式

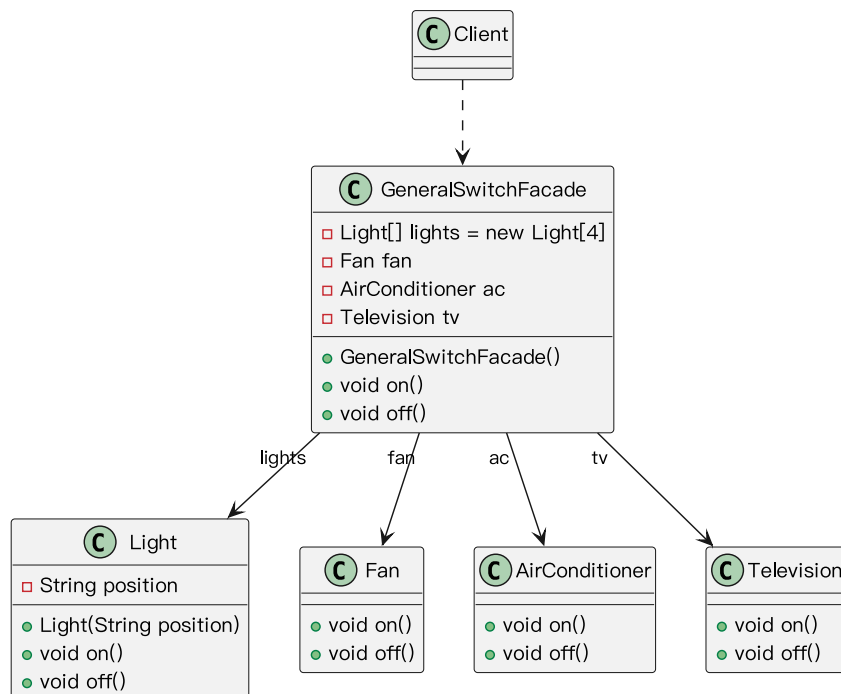
外观模式 (Facade Pattern): 外部与一个子系统的通信必须通过一个统一的外观对象进行, 为子系统的一组接口提供一个一致的界面, 外观模式定义了一个高层接口, 这个接口使得这一子系统更加容易使用。外观模式又称为门面模式, 它是一种对象结构型模式。

外观模式包含如下角色:

- Facade: 外观角色
- SubSystem: 子系统角色



电源总开关: 现在考察一个电源总开关的例子, 以便进一步说明外观模式。为了使用方便, 一个电源总开关可以控制四盏灯、一个风扇、一台空调和一台电视机的启动和关闭。通过该电源总开关可以同时控制上述所有电器设备, 使用外观模式设计该系统。



外观模式的优点:

- 对客户屏蔽子系统组件, 减少了客户处理的对象数目并使得子系统使用起来更加容易。通过引入外观模式, 客户代码将变得很简单, 与之关联的对象也很少。

- 实现了子系统与客户之间的松耦合关系，这使得子系统的组件变化不会影响到调用它的客户类，只需要调整外观类即可。
- 降低了大型软件系统中的编译依赖性，并简化了系统在不同平台之间的移植过程，因为编译一个子系统一般不需要编译所有其他的子系统。一个子系统的修改对其他子系统没有任何影响，而且子系统内部变化也不会影响到外观对象。
- 只是提供了一个访问子系统的统一入口，并不影响用户直接使用子系统类。

外观模式的缺点：

- 不能很好地限制客户使用子系统类，如果对客户访问子系统类做太多的限制则减少了可变性和灵活性。
- 在不引入抽象外观类的情况下，增加新的子系统可能需要修改外观类或客户端的源代码，违背了“开闭原则”。

在以下情况下可以使用外观模式：

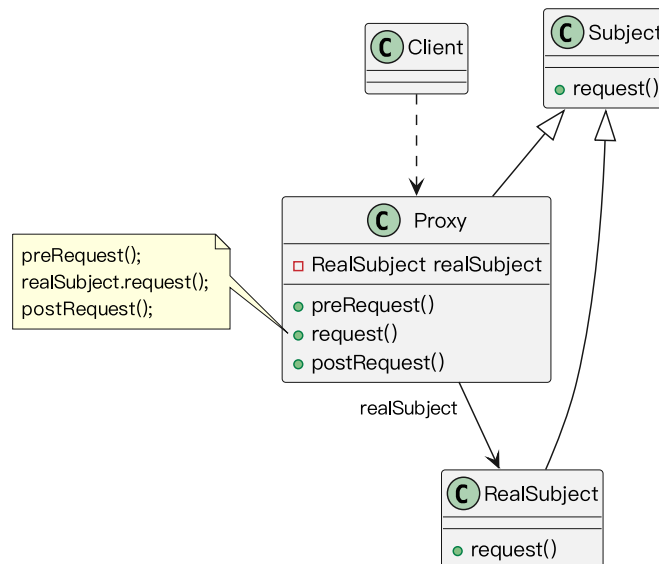
- 当要为一个复杂子系统提供一个简单接口时可以使用外观模式。该接口可以满足大多数用户的需求，而且用户也可以越过外观类直接访问子系统。
- 客户程序与多个子系统之间存在很大的依赖性。引入外观类将子系统与客户以及其他子系统解耦，可以提高子系统的独立性和可移植性。
- 在层次化结构中，可以使用外观模式定义系统中每一层的入口，层与层之间不直接产生联系，而通过外观类建立联系，降低层之间的耦合度。

代理模式

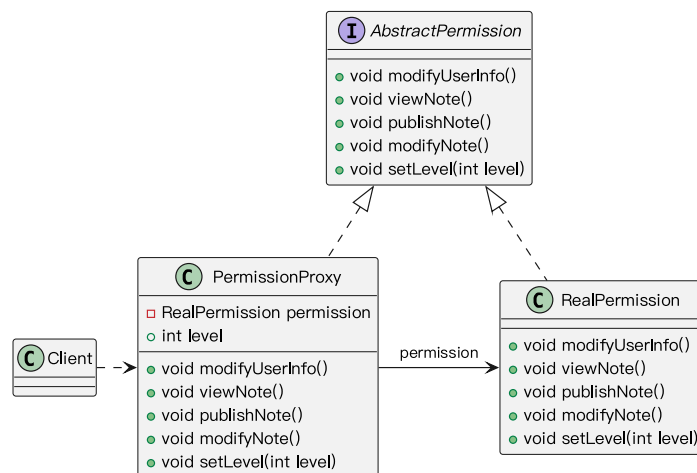
代理模式 (Proxy Pattern)：给某一个对象提供一个代理，并由代理对象控制对原对象的引用。代理模式的英文叫做 Proxy 或 Surrogate，它是一种对象结构型模式。

代理模式包含如下角色：

- Subject：抽象主题角色
- Proxy：代理主题角色
- RealSubject：真实主题角色



论坛权限控制代理：在一个论坛中已注册用户和游客的权限不同，已注册的用户拥有发帖、修改自己的注册信息、修改自己的帖子等功能；而游客只能看到别人发的帖子，没有其他权限。使用代理模式来设计该权限管理模块。在本实例中我们使用代理模式中的保护代理，该代理用于控制对一个对象的访问，可以给不同的用户提供不同级别的使用权限。



代理模式的优点：

- 代理模式能够协调调用者和被调用者，在一定程度上降低了系统的耦合度。
- 远程代理使得客户端可以访问在远程机器上的对象，远程机器可能具有更好的计算性能与处理速度，可以快速响应并处理客户端请求。

- 虚拟代理通过使用一个小对象来代表一个大对象，可以减少系统资源的消耗，对系统进行优化并提高运行速度。
- 保护代理可以控制对真实对象的使用权限。

代理模式的缺点：

- 由于在客户端和真实主题之间增加了代理对象，因此有些类型的代理模式可能会造成请求的处理速度变慢。
- 实现代理模式需要额外的工作，有些代理模式的实现非常复杂。

根据代理模式的使用目的，常见的代理模式有以下几种类型：

- 远程 (Remote) 代理：为一个位于不同的地址空间的对象提供一个本地的代理对象，这个不同的地址空间可以是在同一台主机中，也可是在另一台主机中，远程代理又叫做大使 (Ambassador)。
- 虚拟 (Virtual) 代理：如果需要创建一个资源消耗较大的对象，先创建一个消耗相对较小的对象来表示，真实对象只在需要时才会被真正创建。
- Copy-on-Write 代理：它是虚拟代理的一种，把复制（克隆）操作延迟到只有在客户端真正需要时才执行。一般来说，对象的深克隆是一个开销较大的操作，Copy-on-Write 代理可以让这个操作延迟，只有对象被用到的时候才被克隆。
- 保护 (Protect or Access) 代理：控制对一个对象的访问，可以给不同的用户提供不同级别的使用权限。
- 缓冲 (Cache) 代理：为某一个目标操作的结果提供临时的存储空间，以便多个客户端可以共享这些结果。
- 防火墙 (Firewall) 代理：保护目标不让恶意用户接近。
- 智能引用 (Smart Reference) 代理：当一个对象被引用时，提供一些额外的操作，如将此对象被调用的次数记录下来等。